

torch.svd#

<https://pytorch.org/docs/stable/generated/torch.svd.html#torch.svd>

Description:

Computes the singular value decomposition of either a matrix or batch of matrices input. The singular value decomposition is represented as a namedtuple (U, S, V), such that $\text{input} = U \text{diag}(S) V^H$, where $V^H = U^H \text{diag}(S) V$ is the transpose of V for real inputs, and the conjugate transpose of V for complex inputs. If input is a batch of matrices, then U, S, and V are also batched with the same batch dimensions as input. If some is True (default), the method returns the reduced singular value decomposition. In this case, if the last two dimensions of input are m and n, then the returned U and V matrices will contain only $\min(n, m)$ orthonormal columns. If compute_uv is False, the returned U and V will be zero-filled matrices of shape (m, m) and (n, n) respectively, and the same device as input. The argument some has no effect when compute_uv is False. Supports input of float, double, cfloat and cdouble data types. The dtypes of U and V are the same as input's. S will always be real-valued, even if input is complex.

Function Signature

```
torch.svd(input, some=True, compute_uv=True, *, out=None)#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (tuple, optional) – the output tuple of tensors

Code Examples

```
U, S, Vh = torch.linalg.svd(A, full_matrices=not some)
V = Vh.mH
```

```
S = torch.linalg.svdvals(A)
```

```

>>> a = torch.randn(5, 3)
>>> a
tensor([[ 0.2364, -0.7752,  0.6372],
        [ 1.7201,  0.7394, -0.0504],
        [-0.3371, -1.0584,  0.5296],
        [ 0.3550, -0.4022,  1.5569],
        [ 0.2445, -0.0158,  1.1414]])
>>> u, s, v = torch.svd(a)
>>> u
tensor([[ 0.4027,  0.0287,  0.5434],
        [-0.1946,  0.8833,  0.3679],
        [ 0.4296, -0.2890,  0.5261],
        [ 0.6604,  0.2717, -0.2618],
        [ 0.4234,  0.2481, -0.4733]])
>>> s
tensor([2.3289, 2.0315, 0.7806])
>>> v
tensor([[ -0.0199,  0.8766,  0.4809],
        [-0.5080,  0.4054, -0.7600],
        [ 0.8611,  0.2594, -0.4373]])
>>> torch.dist(a, torch.mm(torch.mm(u, torch.diag(s)), v.t()))
tensor(8.6531e-07)
>>> a_big = torch.randn(7, 5, 3)
>>> u, s, v = torch.svd(a_big)
>>> torch.dist(a_big, torch.matmul(torch.matmul(u,
torch.diag_embed(s)), v.mT))
tensor(2.6503e-06)

```

Notes & Warnings

⚠ WARNING

Warning `torch.svd()` is deprecated in favor of `torch.linalg.svd()` and will be removed in a future PyTorch release. `U, S, V = torch.svd(A, some=some, compute_uv=True)` (default) should be replaced with `U, S, Vh = torch.linalg.svd(A, full_matrices=not some) V = Vh.mH`, `S, _ = torch.svd(A, some=some, compute_uv=False)` should be replaced with `S = torch.linalg.svdvals(A)`

NOTE

Note Differences with `torch.linalg.svd()`: `some` is the opposite of `torch.linalg.svd()`'s `full_matrices`. Note that default value for both is `True`, so the default behavior is effectively the opposite. `torch.svd()` returns `V`, whereas `torch.linalg.svd()` returns `Vh`, that is, VHV^{H} . If `compute_uv` is `False`, `torch.svd()` returns zero-filled tensors for `U` and `Vh`, whereas `torch.linalg.svd()` returns empty tensors.

NOTE

Note The singular values are returned in descending order. If input is a batch of matrices, then the singular values of each matrix in the batch are returned in descending order.

NOTE

Note The `S` tensor can only be used to compute gradients if `compute_uv` is `True`.

NOTE

Note When some is False, the gradients on $U[\dots, :, \min(m, n):]$ and $V[\dots, :, \min(m, n):]$ will be ignored in the backward pass, as those vectors can be arbitrary bases of the corresponding subspaces.

NOTE

Note The implementation of `torch.linalg.svd()` on CPU uses LAPACK's routine `?gesdd` (a divide-and-conquer algorithm) instead of `?gesvd` for speed. Analogously, on GPU, it uses cuSOLVER's routines `gesvdj` and `gesvdjBatched` on CUDA 10.1.243 and later, and MAGMA's routine `gesdd` on earlier versions of CUDA.

NOTE

Note The returned U will not be contiguous. The matrix (or batch of matrices) will be represented as a column-major matrix (i.e. Fortran-contiguous).

⚠ WARNING

Warning The gradients with respect to U and V will only be finite when the input does not have zero nor repeated singular values.

⚠ WARNING

Warning If the distance between any two singular values is close to zero, the gradients with respect to U and V will be numerically unstable, as they depends on $\frac{1}{\min_{i \neq j} |\sigma_i^2 - \sigma_j^2|}$. The same happens when the matrix has small singular values, as these gradients also depend on S^{-1} .

⚠ WARNING

Warning For complex-valued input the singular value decomposition is not unique, as U and V may be multiplied by an arbitrary phase factor $e^{i\phi}$ on every column. The same happens when input has repeated singular values, where one may multiply the columns of the spanning subspace in U and V by a rotation matrix and the resulting vectors will span the same subspace. Different platforms, like NumPy, or inputs on different device types, may produce different U and V tensors.

Detailed Documentation

Documentation

Computes the singular value decomposition of either a matrix or batch of matrices input. The singular value decomposition is represented as a namedtuple (U, S, V) , such that $\text{input} = U \text{diag}(S) V^H = U \text{diag}(S) V^{\text{H}}$ where V^H is the transpose of V for real inputs, and the conjugate transpose of V for complex inputs. If input is a batch of matrices, then U , S , and V are also batched with the same batch dimensions as input.

If `some` is `True` (default), the method returns the reduced singular value decomposition. In this case, if the last two dimensions of input are m and n , then the returned U and V matrices will contain only $\min(n, m)$ orthonormal columns.

If `compute_uv` is `False`, the returned U and V will be zero-filled matrices of shape (m, m) and (n, n) respectively, and the same device as input. The argument `some` has no effect when `compute_uv` is `False`.

Supports input of `float`, `double`, `cfloat` and `cdouble` data types. The dtypes of U and V

are the same as input's. S will always be real-valued, even if input is complex.

`torch.svd()` is deprecated in favor of `torch.linalg.svd()` and will be removed in a future PyTorch release.

`U, S, V = torch.svd(A, some=some, compute_uv=True)` (default) should be replaced with

`_, S, _ = torch.svd(A, some=some, compute_uv=False)` should be replaced with

Differences with `torch.linalg.svd()`:

`some` is the opposite of `torch.linalg.svd()`'s `full_matrices`. Note that default value for both is `True`, so the default behavior is effectively the opposite.

`torch.svd()` returns V , whereas `torch.linalg.svd()` returns V_h , that is, $VH V^{\text{H}}$.

If `compute_uv` is `False`, `torch.svd()` returns zero-filled tensors for U and V_h , whereas `torch.linalg.svd()` returns empty tensors.

The singular values are returned in descending order. If input is a batch of matrices, then the singular values of each matrix in the batch are returned in descending order.

The S tensor can only be used to compute gradients if `compute_uv` is `True`.

When `some` is `False`, the gradients on $U[\dots, :, \min(m, n):]$ and $V[\dots, :, \min(m, n):]$ will be ignored in the backward pass, as those vectors can be arbitrary bases of the corresponding subspaces.

The implementation of `torch.linalg.svd()` on CPU uses LAPACK's routine `?gesdd` (a divide-and-conquer algorithm) instead of `?gesvd` for speed. Analogously, on GPU, it uses cuSOLVER's routines `gesvdj` and `gesvdjBatched` on CUDA 10.1.243 and later, and MAGMA's routine `gesdd` on earlier versions of CUDA.

The returned U will not be contiguous. The matrix (or batch of matrices) will be represented as a column-major matrix (i.e. Fortran-contiguous).

The gradients with respect to U and V will only be finite when the input does not have zero nor repeated singular values.

If the distance between any two singular values is close to zero, the gradients with respect to U and V will be numerically unstable, as they depend on $\frac{1}{\min_{i \neq j} |\sigma_i^2 - \sigma_j^2|}$. The same happens when the matrix has small singular values, as these gradients also depend on S^{-1} .

For complex-valued input the singular value decomposition is not unique, as U and V may be multiplied by an arbitrary phase factor $e^{i\phi}$ on every column. The same happens when input has repeated singular values, where one may multiply the columns of the spanning subspace in U and V by a rotation matrix and the resulting vectors will span the same subspace. Different platforms, like NumPy, or inputs on different device types, may produce different U and V tensors.

`input` (Tensor) – the input tensor of size $(*, m, n)$ where $*$ is zero or more batch dimensions consisting of (m, n) matrices.

`some` (bool, optional) – controls whether to compute the reduced or full decomposition, and consequently, the shape of returned U and V . Default: True.

`compute_uv` (bool, optional) – controls whether to compute U and V . Default: True.

`out` (tuple, optional) – the output tuple of tensors